

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 3
За седьмой семестр
По дисциплине: «Интеллектуальный анализ данных »
Тема: «Предобучение нейронных сетей с использованием автоэнкодерного
подхода»

Выполнил:
Студент 4 курса
Группы ИИ-26
Хомиченко И.А.
Проверил:
Никитин С.К.

Цель: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).
2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.
3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.
4. Выполните пункты 1-3 для датасетов из ЛР 2.
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
7	https://archive.ics.uci.edu/dataset/503/hepatitis+c+virus+hcv+for+egyptian+patients	классификация	Baselinehistological staging

Код программы:

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow.keras import layers, Model
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)
from ucimlrepo import fetch_ucirepo
```

```

np.random.seed(42)
tf.random.set_seed(42)

def build_multiclass_classifier(input_dim, n_classes):
    model = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(n_classes, activation="softmax")
    ])

    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

def build_binary_classifier(input_dim):
    model = tf.keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

def pretrain_autoencoder(data, hidden_dim, epochs=50, batch_size=32):
    inp = layers.Input(shape=(data.shape[1],))

    encoded = layers.Dense(
        hidden_dim,
        activation="relu"
    )(inp)

    decoded = layers.Dense(
        data.shape[1],
        activation="linear"
    )(encoded)

    autoencoder = Model(
        inp,
        decoded
    )

    encoder = Model(
        inp,
        encoded
    )

```

```

autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

autoencoder.fit(
    data,
    data,
    epochs=epochs,
    batch_size=batch_size,
    verbose=0
)

return encoder

def multiclass_report(y_true, y_pred, name):
    accuracy = accuracy_score(
        y_true,
        y_pred
    )

    precision = precision_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    f1_macro = f1_score(
        y_true,
        y_pred,
        average="macro",
        zero_division=0
    )

    f1_weighted = f1_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    print(
        f"{name}: "
        f"Accuracy={accuracy:.4f}, "
        f"Precision={precision:.4f}, "
        f"Recall={recall:.4f}, "
        f"F1_macro={f1_macro:.4f}, "
        f"F1_weighted={f1_weighted:.4f}"
    )

def binary_report(y_true, y_pred, name):
    accuracy = accuracy_score(
        y_true,

```

```

        y_pred
    )

    precision = precision_score(
        y_true,
        y_pred,
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        zero_division=0
    )

    f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0
    )

    print(
        f"{name}: "
        f"Accuracy={accuracy:.4f}, "
        f"Precision={precision:.4f}, "
        f"Recall={recall:.4f}, "
        f"F1={f1:.4f}"
    )

print("\n" + "#" * 80)
print("ЧАСТЬ 1. HEPATITIS C VIRUS FOR EGYPTIAN PATIENTS")
print("#" * 80)

hcv_dataset = fetch_ucirepo(
    id=503
)

X_hcv_df = hcv_dataset.data.features.copy()
y_hcv_df = hcv_dataset.data.targets.copy()

target_name = "Baselinehistological staging"

if target_name not in y_hcv_df.columns:
    matching_columns = [
        col
        for col in y_hcv_df.columns
        if "staging" in col.lower()
    ]

    if len(matching_columns) == 0:
        raise ValueError(
            "Целевая переменная Baselinehistological staging не найдена."
        )

    target_name = matching_columns[0]

y_hcv_raw = y_hcv_df[
    target_name
].values.ravel()

X_hcv_df = X_hcv_df.apply(
    pd.to_numeric,
    errors="coerce"
)

```

```

предобучения показало более высокий F1_macro."
    )
else:
    print(
        "Для HCV обе модели показали одинаковый F1_macro."
    )

fig, axes = plt.subplots(
    1,
    2,
    figsize=(14, 5)
)

sns.heatmap(
    confusion_matrix(
        y_hcv_test,
        y_hcv_baseline
    ),
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[0],
    xticklabels=hcv_class_names,
    yticklabels=hcv_class_names
)

axes[0].set_title(
    "HCV - без предобучения"
)

axes[0].set_xlabel(
    "Predicted"
)

axes[0].set_ylabel(
    "Actual"
)

sns.heatmap(
    confusion_matrix(
        y_hcv_test,
        y_hcv_ae
    ),
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[1],
    xticklabels=hcv_class_names,
    yticklabels=hcv_class_names
)

axes[1].set_title(
    "HCV - AE предобучение"
)

axes[1].set_xlabel(
    "Predicted"
)

axes[1].set_ylabel(
    "Actual"
)

plt.tight_layout()

```

```

plt.savefig(
    "hcv_confusion_matrix.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

plt.figure(
    figsize=(10, 6)
)

plt.plot(
    history_hcv_baseline.history["loss"],
    label="Без предобучения"
)

plt.plot(
    history_hcv_pretrained.history["loss"],
    label="АЕ предобучение"
)

plt.xlabel(
    "Эпоха"
)

plt.ylabel(
    "Loss"
)

plt.title(
    "HCV – сравнение процесса обучения"
)

plt.legend()

plt.grid(
    True,
    alpha=0.3
)

plt.tight_layout()

plt.savefig(
    "hcv_training_loss.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print("\n" + "#" * 80)
print("ЧАСТЬ 2. ПУНКТ 4 – ДАТАСЕТ ИЗ ЛР №2 MUSHROOM")
print("#" * 80)

mushroom_url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/"
    "mushroom/agaricus-lepiota.data"
)

mushroom_columns = [

```

```

        "class",
        "cap-shape",
        "cap-surface",
        "cap-color",
        "bruises",
        "odor",
        "gill-attachment",
        "gill-spacing",
        "gill-size",
        "gill-color",
        "stalk-shape",
        "stalk-root",
        "stalk-surface-above-ring",
        "stalk-surface-below-ring",
        "stalk-color-above-ring",
        "stalk-color-below-ring",
        "veil-type",
        "veil-color",
        "ring-number",
        "ring-type",
        "spore-print-color",
        "population",
        "habitat"
    ]

mushroom_df = pd.read_csv(
    mushroom_url,
    header=None,
    names=mushroom_columns
)

print(
    "Размер Mushroom:",
    mushroom_df.shape
)

print(
    "\nРаспределение классов:"
)

print(
    mushroom_df["class"].value_counts()
)

X_mushroom_df = mushroom_df.drop(
    columns=["class"]
)

y_mushroom_raw = mushroom_df[
    "class"
].values

y_mushroom = np.array([
    1 if value == "p" else 0
    for value in y_mushroom_raw
])

X_mush_train_df, X_mush_test_df, y_mush_train, y_mush_test =
train_test_split(
    X_mushroom_df,
    y_mushroom,
    test_size=0.2,
    random_state=42,
    stratify=y_mushroom
)

```

```

)

try:
    mushroom_encoder = OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    )
except TypeError:
    mushroom_encoder = OneHotEncoder(
        handle_unknown="ignore",
        sparse=False
    )

X_mush_train_encoded = mushroom_encoder.fit_transform(
    X_mush_train_df
)

X_mush_test_encoded = mushroom_encoder.transform(
    X_mush_test_df
)

mushroom_scaler = StandardScaler()

X_mush_train = mushroom_scaler.fit_transform(
    X_mush_train_encoded
).astype(np.float32)

X_mush_test = mushroom_scaler.transform(
    X_mush_test_encoded
).astype(np.float32)

mushroom_input_dim = X_mush_train.shape[1]

print(
    "\nКоличество признаков после One-Hot Encoding:",
    mushroom_input_dim
)

print(
    "Количество объектов обучающей выборки:",
    X_mush_train.shape[0]
)

print(
    "Количество объектов тестовой выборки:",
    X_mush_test.shape[0]
)

print("\n" + "=" * 80)
print("MUSHROOM – ОБУЧЕНИЕ БЕЗ ПРЕДОБУЧЕНИЯ")
print("=" * 80)

mushroom_baseline = build_binary_classifier(
    mushroom_input_dim
)

history_mushroom_baseline = mushroom_baseline.fit(
    X_mush_train,
    y_mush_train,
    epochs=100,
    batch_size=64,
    validation_split=0.1,
    verbose=0
)

```

```

y_mushroom_baseline_prob = mushroom_baseline.predict(
    X_mush_test,
    verbose=0
).ravel()

y_mushroom_baseline = (
    y_mushroom_baseline_prob >= 0.5
).astype(int)

binary_report(
    y_mush_test,
    y_mushroom_baseline,
    "Mushroom без предобучения"
)

print("\n" + "=" * 80)
print("MUSHROOM — АВТОЭНКODЕРНОЕ ПРЕДОБУЧЕНИЕ")
print("=" * 80)

print("Предобучение слоя 1: 64 нейрона")

mushroom_ae1 = pretrain_autoencoder(
    X_mush_train,
    hidden_dim=64,
    epochs=50,
    batch_size=64
)

H1_mushroom_train = mushroom_ae1.predict(
    X_mush_train,
    verbose=0
)

print("Предобучение слоя 2: 32 нейрона")

mushroom_ae2 = pretrain_autoencoder(
    H1_mushroom_train,
    hidden_dim=32,
    epochs=50,
    batch_size=64
)

H2_mushroom_train = mushroom_ae2.predict(
    H1_mushroom_train,
    verbose=0
)

print("Предобучение слоя 3: 16 нейронов")

mushroom_ae3 = pretrain_autoencoder(
    H2_mushroom_train,
    hidden_dim=16,
    epochs=50,
    batch_size=64
)

mushroom_pretrained = build_binary_classifier(
    mushroom_input_dim
)

mushroom_pretrained.layers[0].set_weights(
    mushroom_ae1.get_weights()
)

```

```

mushroom_pretrained.layers[1].set_weights(
    mushroom_ae2.get_weights()
)

mushroom_pretrained.layers[2].set_weights(
    mushroom_ae3.get_weights()
)

history_mushroom_pretrained = mushroom_pretrained.fit(
    X_mush_train,
    y_mush_train,
    epochs=100,
    batch_size=64,
    validation_split=0.1,
    verbose=0
)

y_mushroom_ae_prob = mushroom_pretrained.predict(
    X_mush_test,
    verbose=0
).ravel()

y_mushroom_ae = (
    y_mushroom_ae_prob >= 0.5
).astype(int)

binary_report(
    y_mush_test,
    y_mushroom_ae,
    "Mushroom с АЕ-предобучением"
)

print("\n" + "=" * 80)
print("MUSHROOM — СПАВНЕНИЕ")
print("=" * 80)

mushroom_comparison = pd.DataFrame({
    "Метрика": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1"
    ],
    "Без предобучения": [
        accuracy_score(
            y_mush_test,
            y_mushroom_baseline
        ),
        precision_score(
            y_mush_test,
            y_mushroom_baseline,
            zero_division=0
        ),
        recall_score(
            y_mush_test,
            y_mushroom_baseline,
            zero_division=0
        ),
        f1_score(
            y_mush_test,
            y_mushroom_baseline,
            zero_division=0
        )
    ]
})

```

```

    ],
    "С АЕ-предобучением": [
        accuracy_score(
            y_mush_test,
            y_mushroom_ae
        ),
        precision_score(
            y_mush_test,
            y_mushroom_ae,
            zero_division=0
        ),
        recall_score(
            y_mush_test,
            y_mushroom_ae,
            zero_division=0
        ),
        f1_score(
            y_mush_test,
            y_mushroom_ae,
            zero_division=0
        )
    ]
})

mushroom_comparison[
    ["Без предобучения", "С АЕ-предобучением"]
] = mushroom_comparison[
    ["Без предобучения", "С АЕ-предобучением"]
].round(4)

mushroom_comparison["Разница"] = (
    mushroom_comparison["С АЕ-предобучением"]
    - mushroom_comparison["Без предобучения"]
).round(4)

print(
    mushroom_comparison.to_string(
        index=False
    )
)

mushroom_baseline_f1 = f1_score(
    y_mush_test,
    y_mushroom_baseline,
    zero_division=0
)

mushroom_ae_f1 = f1_score(
    y_mush_test,
    y_mushroom_ae,
    zero_division=0
)

mushroom_delta_f1 = (
    mushroom_ae_f1
    - mushroom_baseline_f1
)

print(
    f"\nMushroom F1 без предобучения: "
    f"{mushroom_baseline_f1:.4f}"
)

print(

```

```

        f"Mushroom F1 с AE-предобучением: "
        f"{mushroom_ae_f1:.4f}"
    )

print(
    f"Изменение F1: "
    f"{mushroom_delta_f1:+.4f}"
)

if mushroom_delta_f1 > 0:
    print(
        "Для Mushroom автоэнкодерное предобучение улучшило F1."
    )
elif mushroom_delta_f1 < 0:
    print(
        "Для Mushroom обучение без предобучения показало более высокий F1."
    )
else:
    print(
        "Для Mushroom обе модели показали одинаковый F1."
    )

fig, axes = plt.subplots(
    1,
    2,
    figsize=(12, 5)
)

sns.heatmap(
    confusion_matrix(
        y_mush_test,
        y_mushroom_baseline
    ),
    annot=True,
    fmt="d",
    cmap="Blues",
    ax=axes[0],
    xticklabels=[
        "Edible",
        "Poisonous"
    ],
    yticklabels=[
        "Edible",
        "Poisonous"
    ]
)

axes[0].set_title(
    "Mushroom – без предобучения"
)

axes[0].set_xlabel(
    "Predicted"
)

axes[0].set_ylabel(
    "Actual"
)

sns.heatmap(
    confusion_matrix(
        y_mush_test,
        y_mushroom_ae
    ),

```

```

        annot=True,
        fmt="d",
        cmap="Blues",
        ax=axes[1],
        xticklabels=[
            "Edible",
            "Poisonous"
        ],
        yticklabels=[
            "Edible",
            "Poisonous"
        ]
    )

axes[1].set_title(
    "Mushroom - AE предобучение"
)

axes[1].set_xlabel(
    "Predicted"
)

axes[1].set_ylabel(
    "Actual"
)

plt.tight_layout()

plt.savefig(
    "mushroom_confusion_matrix.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

plt.figure(
    figsize=(10, 6)
)

plt.plot(
    history_mushroom_baseline.history["loss"],
    label="Без предобучения"
)

plt.plot(
    history_mushroom_pretrained.history["loss"],
    label="AE предобучение"
)

plt.xlabel(
    "Эпоха"
)

plt.ylabel(
    "Loss"
)

plt.title(
    "Mushroom - сравнение процесса обучения"
)

plt.legend()

```

```

plt.grid(
    True,
    alpha=0.3
)

plt.tight_layout()

plt.savefig(
    "mushroom_training_loss.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print("\n" + "#" * 80)
print("ОБЩИЕ ИТОГИ")
print("#" * 80)

print(
    f"\nHCV:"
)

print(
    f"F1_macro без предобучения = "
    f"{hcv_baseline_f1:.4f}"
)

print(
    f"F1_macro с AE = "
    f"{hcv_ae_f1:.4f}"
)

print(
    f"Разница = "
    f"{hcv_delta_f1:+.4f}"
)

print(
    f"\nMushroom:"
)

print(
    f"F1 без предобучения = "
    f"{mushroom_baseline_f1:.4f}"
)

print(
    f"F1 с AE = "
    f"{mushroom_ae_f1:.4f}"
)

print(
    f"Разница = "
    f"{mushroom_delta_f1:+.4f}"
)

print("\nВсе задания выполнены.")
print("1. Обучение без предобучения выполнено.")
print("2. Автоэнкодерное предобучение выполнено.")
print("3. Сравнение результатов выполнено.")
print("4. Пункты 1-3 выполнены для датасета Mushroom из ЛР №2.")

```

Результат программы:

ЧАСТЬ 1. HEPATITIS C VIRUS FOR EGYPTIAN PATIENTS

Размер датасета: (1385, 28)

Количество классов: 4

Классы: ['1', '2', '3', '4']

Распределение классов:

Класс 1: 336

Класс 2: 332

Класс 3: 355

Класс 4: 362

HCV — ОБУЧЕНИЕ БЕЗ ПРЕДОБУЧЕНИЯ

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

HCV без предобучения: Accuracy=0.2166, Precision=0.2226, Recall=0.2166, F1_macro=0.2164, F1_weighted=0.2166

HCV — АВТОЭНКODЕРНОЕ ПРЕДОБУЧЕНИЕ

Предобучение слоя 1: 64 нейрона

Предобучение слоя 2: 32 нейрона

Предобучение слоя 3: 16 нейронов

HCV с АЕ-предобучением: Accuracy=0.2563, Precision=0.2575, Recall=0.2563, F1_macro=0.2451, F1_weighted=0.2464

HCV — СРАВНЕНИЕ

Метрика	Без предобучения	С АЕ-предобучением	Разница
Accuracy	0.2166	0.2563	0.0397
Precision weighted	0.2226	0.2575	0.0349
Recall weighted	0.2166	0.2563	0.0397

F1 macro	0.2164	0.2451	0.0287
F1 weighted	0.2166	0.2464	0.0298

HCV F1_macro без предобучения: 0.2164

HCV F1_macro с AE-предобучением: 0.2451

Изменение F1_macro: +0.0287

Для HCV автоэнкодерное предобучение улучшило F1_macro.

ЧАСТЬ 2. DATASET ИЗ ЛР №2 MUSHROOM

Размер Mushroom: (8124, 23)

Распределение классов:

class

e 4208

p 3916

Name: count, dtype: int64

Количество признаков после One-Hot Encoding: 117

Количество объектов обучающей выборки: 6499

Количество объектов тестовой выборки: 1625

MUSHROOM — ОБУЧЕНИЕ БЕЗ ПРЕДОБУЧЕНИЯ

Mushroom без предобучения: Accuracy=1.0000, Precision=1.0000, Recall=1.0000, F1=1.0000

MUSHROOM — АВТОЭНКОДЕРНОЕ ПРЕДОБУЧЕНИЕ

Предобучение слоя 1: 64 нейрона

Предобучение слоя 2: 32 нейрона

Предобучение слоя 3: 16 нейронов

Mushroom с AE-предобучением: Accuracy=1.0000, Precision=1.0000, Recall=1.0000, F1=1.0000

MUSHROOM — СРАВНЕНИЕ

Метрика	Без предобучения	С АЕ-предобучением	Разница
Accuracy	1.0	1.0	0.0
Precision	1.0	1.0	0.0
Recall	1.0	1.0	0.0
F1	1.0	1.0	0.0

Mushroom F1 без предобучения: 1.0000

Mushroom F1 с АЕ-предобучением: 1.0000

Изменение F1: +0.0000

Для Mushroom обе модели показали одинаковый F1.

ОБЩИЕ ИТОГИ

HCV:

F1_macro без предобучения = 0.2164

F1_macro с АЕ = 0.2451

Разница = +0.0287

Mushroom:

F1 без предобучения = 1.0000

F1 с АЕ = 1.0000

Разница = +0.0000

Figure 1

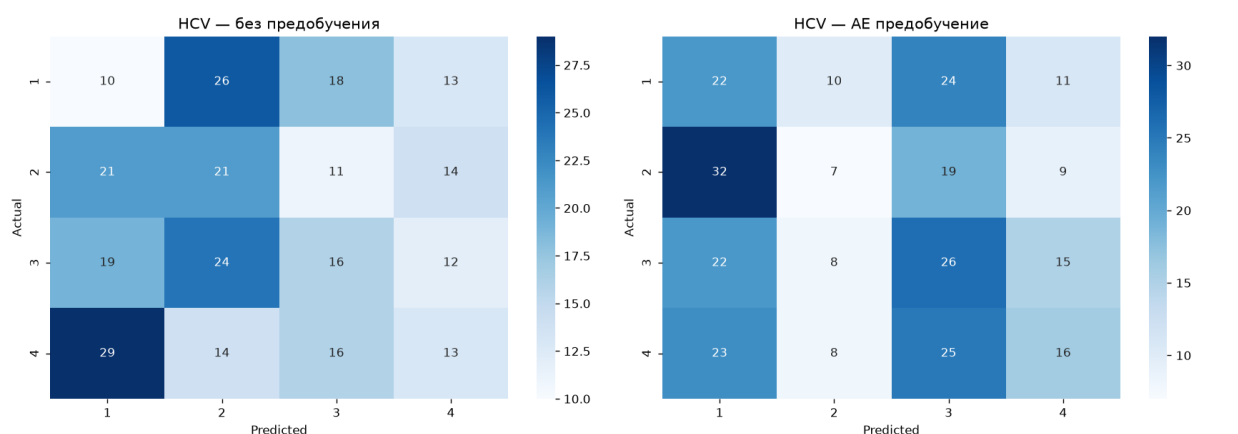


Figure 1

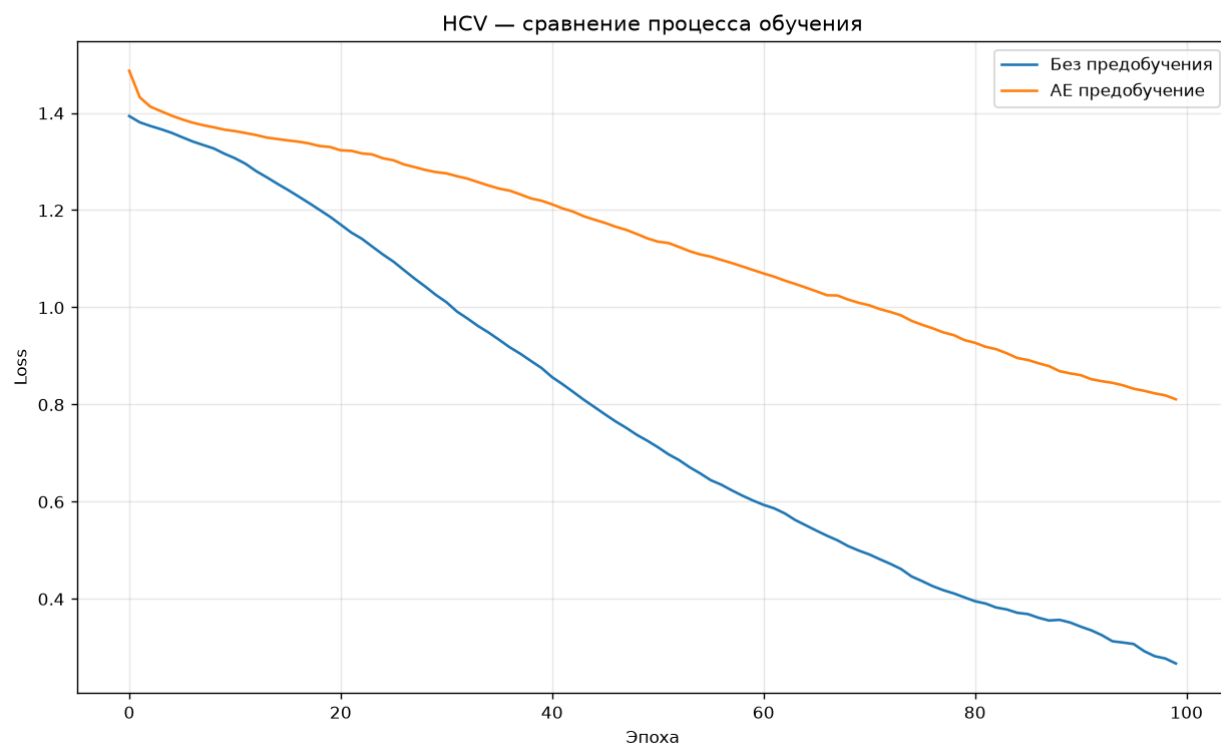


Figure 1

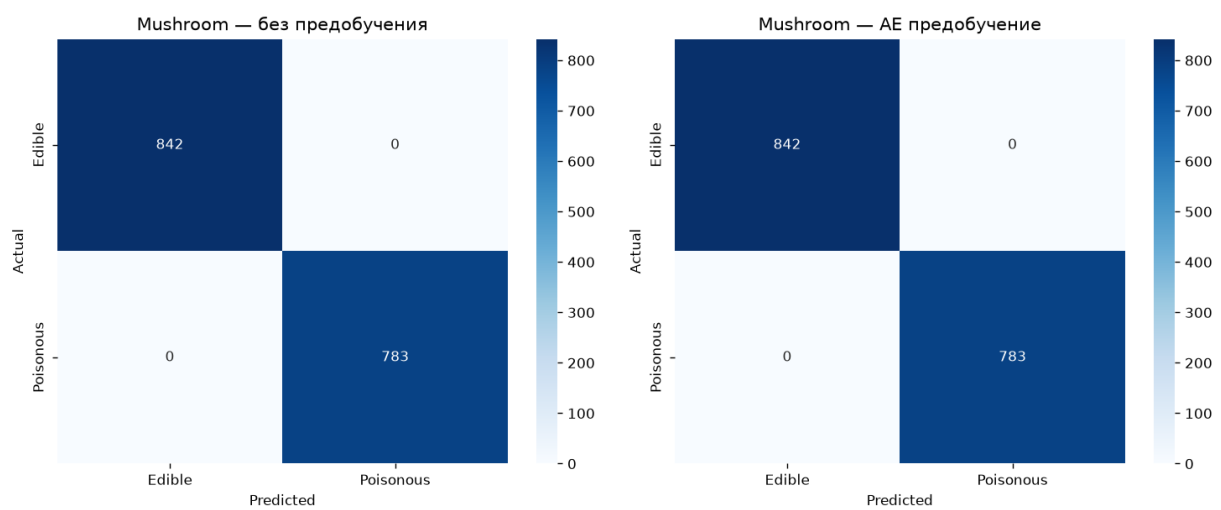


Figure 1



Вывод: научился осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода.